KSE | Kyiv School of Economics

Applied Linear Algebra

Autumn 2026/2027

LAB 1. LINEAR TRANSFORMATIONS

Write a program that implements linear transformations.

GENERAL INFORMATION

- The recommended programming language is Python. Note that other programming languages may not provide sufficient functionality to complete all parts.
- You must write the program code yourself.
- In this assignment you are only allowed to use NumPy for working with matrices and Matplotlib, or similar libraries, for data visualization. You are not allowed to use any pre-built functions for linear transformations from external libraries.

THEORETICAL BACKGROUND

A *linear transformation* is a mapping between vector spaces that preserves vector addition and scalar multiplication.

More precisely, let L_n and L_m be vector spaces. A map $\varphi\colon L_n \to L_m$ is a linear transformation from L_n into L_m if

$$\varphi(\mathbf{x}+\mathbf{y}) = \varphi\mathbf{x} + \varphi\mathbf{y} \text{ and } \varphi\lambda\mathbf{x} = \lambda\varphi\mathbf{x}\,.$$

for all vectors $\mathbf{x}, \mathbf{y}$ from L_n and scalars $\lambda \in \mathbb{R}$.

Every linear transformation φ from an n-dimensional vector space into an m-dimensional vector space can be represented by an $m \times n$ matrix $\mathbf{A}$, called the *matrix representation* of φ. The action of a linear mapping on a vector $\mathbf{x} \in L_n$ is represented by multiplying its matrix by the column coordinate of the vector $\mathbf{x}$. To apply φ to a vector $\mathbf{x}$, left multiply by its matrix representation. This results in a new vector $\mathbf{y} \in L_m$, where each component is some linear combination of the elements of $\mathbf{x}$. For linear transformations from $\mathbb{R}^2$ to $\mathbb{R}^2$, this process has the form

$$\mathbf{Ax} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} ax_1 + bx_2 \\ cx_1 + dx_2 \end{pmatrix} = \begin{pmatrix} y_1 \\ y_2 \end{pmatrix} = \mathbf{y}\,.$$



Linear transformations can be interpreted geometrically.

To demonstrate this, consider an array $\mathbf{L}$ of points that collectively form the silhouette of a picture of the KSE Lynx above. The coordinate pairs are organized by column, so the array has two rows: one for x-coordinates, and one for y-coordinates. Matrix multiplication on the left transforms each coordinate pair, resulting in another matrix $\mathbf{L}'$ whose columns are the transformed coordinate pairs:

$$\mathbf{AL} = \mathbf{A} \begin{pmatrix} x_1 & x_2 & \cdots & x_k \\ y_2 & y_2 & \cdots & y_k \end{pmatrix} = \begin{pmatrix} x_1' & x_2' & \cdots & x_k' \\ y_2' & y_2' & \cdots & y_k' \end{pmatrix} = \mathbf{L}'$$

Types of Linear Transformations

Linear transformations from $\mathbb{R}^2$ into $\mathbb{R}^2$ can be classified in a few ways.

- **Stretch**: Stretches or compresses the vector along each axis. The matrix representation is diagonal:

  $$\begin{pmatrix} a & 0 \\ 0 & b \end{pmatrix}$$

  If $a = b$, the transformation is called a *dilation*.

- **Shear**: Slants the vector by a scalar factor horizontally or vertically (or both simultaneously). The matrix representation is

  $$\begin{pmatrix} 1 & a \\ b & 1 \end{pmatrix}.$$

  Pure horizontal shears ($b = 0$) skew the -coordinate of the vector while pure vertical shears ($a = 0$) skew the -coordinate.

- **Reflection**: Reflects the vector about a line that passes through the origin. The reflection about the line spanned by the vector $(a, b)^T$ has the matrix representation

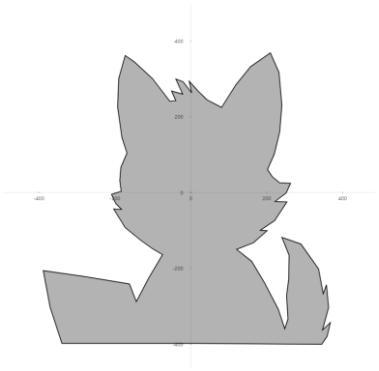
$$\frac{1}{a^2+b^2}\begin{pmatrix} a^2-b^2 & 2ab \\ 2ab & b^2-a^2 \end{pmatrix}$$

- **Rotation**: Rotates the vector around the origin. A counterclockwise rotation of θ radians has the following matrix representation:

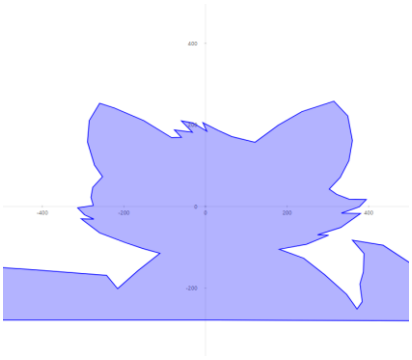
$$\begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

A negative value of θ performs a clockwise rotation. Choosing $\theta = \frac{\pi}{2}$ produces a rotation of 90^0 counterclockwise.

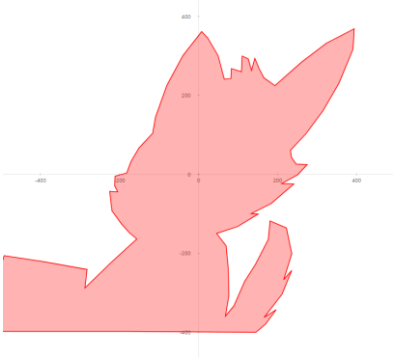
Below is an example of each of these linear transformations.



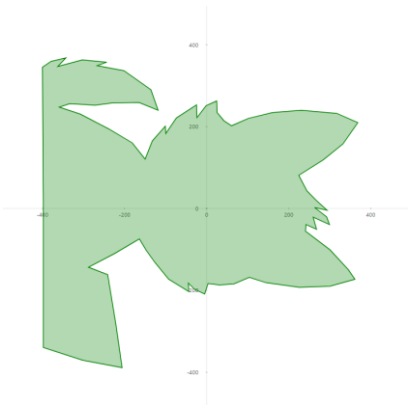
Original



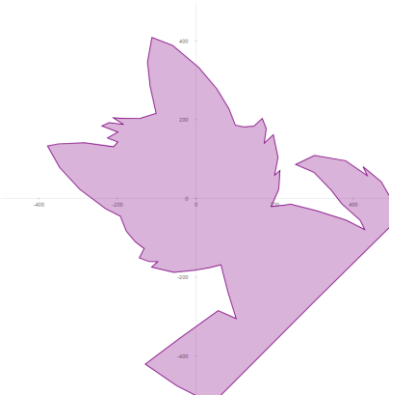
Stretch (1.5, 0.7)



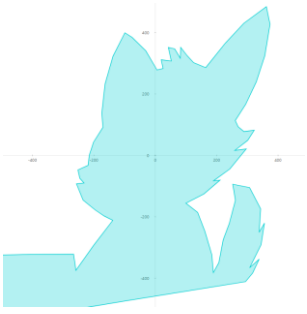
Shear (0.5, 0)



Reflection (line xy)



Rotation (45^0)



Composition (Stretch+Sher+Rotation)

Compositions of Linear Transformations

Let L_n, L_m and L_p be finite-dimensional vector spaces.

If $\psi: L_n \to L_m$ and $\varphi: L_m \to L_p$ are linear transformations with matrix representations $\mathbf{A}$ and $\mathbf{B}$, respectively, then the *composition* function $\varphi \cdot \psi : L_n \to L_p$ is also a linear transformation, and its matrix representation is the matrix product $\mathbf{BA}$.

For example, if $\mathbf{S}$ is a matrix representing a shear and $\mathbf{R}$ is a matrix representing a rotation, then $\mathbf{RS}$ represents a shear followed by a rotation. In fact, any linear transformation $\varphi: \mathbb{R}^2 \to \mathbb{R}^2$ can be written as a composition of the four transformations.

Linear Transformations in 3-D

In the same way that linear transformations from $\mathbb{R}^2$ into $\mathbb{R}^2$ can be represented by 2×2 matrices, linear transformations from $\mathbb{R}^3$ into $\mathbb{R}^3$ can be represented by 3×3 matrices. We will simply consider the rotation transformation on a 3D model of an airplane

The figure shows a 3D model of an airplane.



This is what the data points look like plotted with 50,000 points in 3-D.

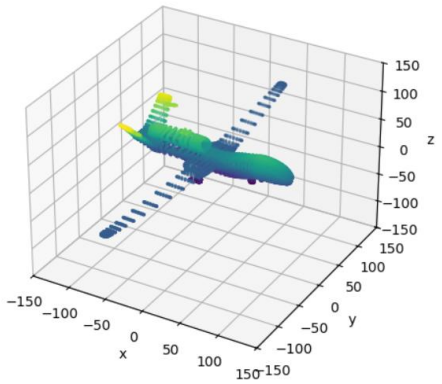


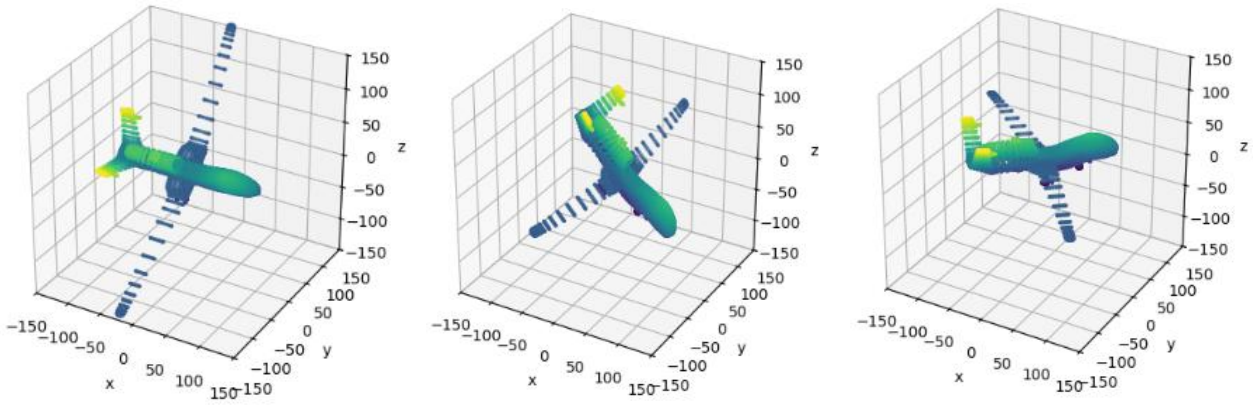
Figure by EMC2 at Brigham Young University

While in 2-D we could only rotate on the xy plane, in 3-D we can rotate on the xy plane, the yz plane, and the xz plane making 3 different rotations. With all 3 of these rotations, we can achieve any rotation in 3-D. Here is what the 3 different rotations look like.

Here are the matrix representations for each of these 3 kinds of rotations in $\mathbb{R}^3$:

- xy plane rotation: $\begin{pmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$
- yz plane rotation: $\begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{pmatrix}$
- xz plane rotation: $\begin{pmatrix} \cos\theta & 0 & -\sin\theta \\ 0 & 1 & 0 \\ \sin\theta & 0 & \cos\theta \end{pmatrix}$

Below is an example of each of the 3 rotations.



Rotation 45^0 about x-axis Rotation 45^0 about y-axis Rotation 45^0 about z-axis

Figure by EMC2 at Brigham Young University

TASKS

Part 1

Create and visualize an object that displays linear transformations in two-dimensional space (it is advisable to avoid symmetrical images so that transformations are more noticeable).

Task 1.

Write a function for each type of linear transformation:
`stretch(X,a,b)`, `shear(X,a,b)`, `reflection(X,a,b)`, `rotation(X,`θ`)`.
Each function should accept an array to transform and the scalars that define the transformation (a, b for stretch, shear, and reflection, and θ for rotation). Construct the matrix representation, multiply it with the input array, and return a transformation of the data. Make sure to copy the array before transforming it.
After each transformation, you must output the resulting image and its matrix.
By modifying the transformation matrices, investigate how different matrix elements affect the result

Task 2.

Take the data points and apply a combination of Stretch, Sher and Rotation.
Change the order of transformations several (at least 3) times. Does the final result depend on the order of transformations?
After each transformation, you must output the resulting image and its matrix.

Part 2

For this part of the lab we will be using data points from the

https://www.kaggle.com/datasets/balraj98/modelnet40-princeton-3d-object-dataset

Task 3

Write a function for each 3-D rotation (`rotate_xy (X,`θ`)`, `rotate_yz (X,`θ`)`, `rotate_xz (X,` θ`)`). Each function should accept an array to transform and the angle for the rotation. Construct the matrix representation, multiply it with the input array, and return a transformation of the data. Make sure to copy the array before transforming it.
After each transformation you must output the resulting image and its matrix.

Task 4.

Apply to the data points a combination of these 3 rotations in any order.
Output the resulting image and matrix of transformation.

LAB GRADING

Correctness/quality of the code 6 points

Tasks 1 and 3 - 2 points for each task

Tasks 2 and 4 - 1 point for each task

The maximum mark for each assignment is given if the program works correctly and the results of its work fully meet the requirements of the task.

Laboratory work defense 4 points

The maximum mark is given if you can explain the results of each program, analyze them, and draw appropriate conclusions.

Theoretical questions 2 points

1. **0.25**pts What are linear transformations?

2. **0.25**pts What is a linear transformation matrix and how can it be interpreted?

3. **0.25**pts What features and properties does a rotation matrix have?

4. **0.25**pts Suppose some arbitrary linear transformation has been performed; how do you find the linear transformation matrix that will return everything to its original form? Is it always possible to perform an inverse transformation?

4. **1**pts The absolute value of the determinant of a transformation matrix is less than 1, what conclusions can be drawn about this transformation (how does space change under this transformation)? What if it is greater than 1? Equal to 1? Equal to 0?

USEFUL MATERIALS AND LINKS

Video 1, video 2 that will help refresh your knowledge of linear transformations

Simple 2D model example

batman = np.array([[0, 0], [1, 0.2], [0.4, 1], [0.5, 0.4], [0, 0.8], [-0.5, 0.4], [-0.4, 1], [-1, 0.2], [0, 0]])

To build a 2D plot of a KSE Lynx silhouette and obtain its dataset this resource was used.

lynx = np.array([

[209.70, 368.42], [157.63, 332.16], [118.82, 284.21], [80.95, 224.56], [43.08, 244.44], [20.36, 266.67], [-4.26, 293.57], [2.37, 263.16], [-20.36, 292.40], [-39.29, 299.42], [-21.30, 259.65], [-50.65, 267.84], [-39.29, 242.11], [-55.38, 240.94], [-100.83, 300.58], [-149.11, 345.03], [-172.78, 361.40], [-189.82, 300.58], [-192.66, 225.73], [-181.30, 145.03], [-168.05, 104.09], [-184.14, 66.67], [-186.98, 31.58], [-183.20, 3.51], [-208.76, -4.68], [-197.40, -29.24], [-182.25, -44.44], [-203.08, -43.27], [-172.78, -92.40], [-131.12, -126.32], [-101.78, -147.37], [-74.32, -163.74], [-110.30, -224.56], [-143.43, -287.72], [-161.42, -240.94], [-282.60, -221.05], [-388.64, -205.85], [-370.65, -301.75], [-339.41, -397.66], [18.46, -397.66], [345.09, -400.00], [359.29, -378.95], [367.81, -342.69], [346.98, -362.57], [363.08, -302.92], [357.40, -243.27], [348.88, -266.67], [336.57, -201.17], [290.18, -135.67], [240.00, -118.13], [258.93, -164.91], [257.99, -228.07], [252.31, -271.35], [256.09, -333.33], [247.57, -359.06], [230.53, -307.60], [194.56, -238.60], [160.47, -181.29], [120.71, -149.71], [165.21, -132.16], [201.18, -100.58], [183.20, -99.42], [221.07, -73.68], [253.25, -24.56], [222.01, -23.39], [251.36, -1.17], [262.72, 24.56], [234.32, 25.73], [214.44, 42.11], [202.13, 60.82], [220.12, 101.75], [234.32, 160.23], [240.00, 230.41], [232.43, 316.96]

])

To read the image

image = cv2.imread('image.jpg')

The following provided by **Vasyl Moisieiev** AI27

Articles about .off files

https://shape.cs.princeton.edu/benchmark/documentation/off_format.html

https://en.wikipedia.org/wiki/OFF_%28file_format%29

Code for reading files and visualizations

Reading a file

```
def read_off(filename: str):
    with open(filename, 'r') as f:
        # Перевіряємо, чи перший рядок починається з OFF
        if 'OFF' != f.readline().strip():
            raise ValueError('Not a valid OFF header')

        # Зчитуємо кількість вершин, граней та ребер (третє значення часто
ігнорується)
        n_verts, n_faces, _ = map(int, f.readline().strip().split())

        # Зчитуємо координати всіх вершин (x, y, z)
        verts = [list(map(float, f.readline().strip().split())) for _ in
range(n_verts)]

        # Зчитуємо грані: перше число у рядку - кількість вершин грані (ігноруємо),
далі індекси
        faces = [list(map(int, f.readline().strip().split()[1:])) for _ in
range(n_faces)]

    # Повертаємо вершини у вигляді масиву NumPy та список граней
    return np.array(verts), faces
```

File reading example

```
vertices, faces = read_off("/Users/admin/Documents/GitHub/marketing-
utils/tests/ModelNet40 Dataset/ModelNet40/airplane/test/airplane_0627.off")
print(vertices.shape)
print(faces[:3])
print(vertices)
```

Visualization using matplotlib

```
# Імпортуємо matplotlib для побудови 3D-графіків
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection

# Функція для візуалізації OFF-моделі у вигляді сітки та точок
# (зручно бачити, як змінюється модель після трансформацій)
def plot_off(vertices, faces):
    fig = plt.figure(figsize=(8, 8))                    # створюємо вікно
    ax = fig.add_subplot(111, projection='3d')          # додаємо 3D координатну систему

    # Створюємо полігональну сітку з граней (faces) та додаємо її на графік
    mesh = Poly3DCollection([vertices[face] for face in faces],
                            alpha=0.3, edgecolor='k')  # прозорість 0.3, чорні ребра
    ax.add_collection3d(mesh)

    # Додаємо вершини як червоні точки
    ax.scatter(vertices[:, 0], vertices[:, 1], vertices[:, 2], s=2, c='r')

    # Підписуємо осі
    ax.set_xlabel("X")
    ax.set_ylabel("Y")
    ax.set_zlabel("Z")

    # Автоматично масштабуємо сцену під модель
    ax.auto_scale_xyz(vertices[:, 0], vertices[:, 1], vertices[:, 2])
```

```
    plt.show()  # показуємо результат

# Виклик функції для побудови OFF-моделі
plot_off(vertices, faces)
```

Interactive visualization to view the model

```
# --- Тепер альтернативна візуалізація через Open3D --- #
import open3d as o3d

# Функція для інтерактивного перегляду 3D-моделі
# (можна крутити, наближати та віддаляти мишею)
def visualize_off(vertices, faces):
    mesh = o3d.geometry.TriangleMesh()                 # створюємо трикутну сітку
    mesh.vertices = o3d.utility.Vector3dVector(vertices) # завантажуємо вершини
    mesh.triangles = o3d.utility.Vector3iVector(faces)  # завантажуємо індекси граней
    mesh.compute_vertex_normals()                       # рахуємо нормалі для коректного відображення

    o3d.visualization.draw_geometries([mesh])          # відкриваємо інтерактивне вікно

# Виклик для перегляду
visualize_off(vertices, faces)
```